

数据集

数据集为 University of Minnesota 提供的开源文件。

ratings.dat

所有评分存储在文件“ratings.dat”中，格式如下：

UserID::MovieID::Rating::Timestamp

- 用户 ID 范围为 1 至 6040
- 电影 ID 范围为 1 至 3952
- 评分为 5 星制（仅整数评分）
- 时间戳以自 Unix 时间纪元起的秒数表示
- 每位用户至少提供 20 个评分

users.dat

用户信息存储在文件“users.dat”中，格式如下：

UserID::Gender::Age::Occupation::Zip-code

所有用户信息均由用户自愿提供，未核实其准确性。仅包含提供部分人口统计信息的用户。

- 性别以“M”表示男性，“F”表示女性
- 年龄范围如下：
 - 1: “18 岁以下”
 - 18: “18-24 岁”
 - 25: “25-34 岁”
 - 35: “35-44 岁”
 - 45: “45-49 岁”
 - 50: “50-55 岁”
 - 56: “56 岁及以上”
- 职业类别如下：
 - 0: “其他”或未指定
 - 1: “学术/教育工作者”
 - 2: “艺术家”
 - 3: “文职/行政”
 - 4: “大学/研究生”
 - 5: “客户服务”
 - 6: “医生/医疗”
 - 7: “高管/管理”
 - 8: “农民”
 - 9: “家庭主妇”
 - 10: “中小学生”
 - 11: “律师”
 - 12: “程序员”
 - 13: “退休”
 - 14: “销售/市场营销”
 - 15: “科学家”
 - 16: “自雇人士”

- 17: “技术员/工程师”
- 18: “工匠”
- 19: “无业”
- 20: “作家”

movies.dat

电影信息存储在文件“movies.dat”中，格式如下：

MovieID::Title::Genres

- 电影标题与 IMDb 提供的标题一致（包括发行年份）
- 类型用管道符（|）分隔，选自以下类型：
 - 动作
 - 冒险
 - 动画
 - 儿童
 - 喜剧
 - 犯罪
 - 纪录片
 - 剧情
 - 奇幻
 - 黑色电影
 - 恐怖
 - 音乐剧
 - 悬疑
 - 爱情
 - 科幻
 - 惊悚
 - 战争
 - 西部
- 由于意外的重复条目和/或测试条目，某些电影 ID 并不对应实际电影
- 电影信息大多为手动输入，因此可能存在错误和不一致

任务

基于用户的协同过滤算法

基于用户的协同过滤算法即找到与其相似的用户，推荐相似用户喜欢的物品。

主要包括以下两个步骤：

- 计算用户之间的**余弦相似度**，找出与目标用户兴趣相似的用户集合。
- 根据用户**余弦相似度**及其对物品的评价，为物品进行评分，并将该集合中用户喜欢且目标用户尚未了解的物品推荐给目标用户。

实现步骤

1. 读取数据并做出切分。

2. 定义精确率，召回率，覆盖率，新颖度作为评估指标。

1. 精确率 (Precision)

- **定义：**
推荐系统推荐的物品中，实际被用户接受的物品比例。
反映推荐结果的准确性。

- **公式：**

$$\text{Precision} = \frac{\text{命中数}}{\text{推荐总数}}$$

示例代码

```
1  # 定义精确率指标计算方式
2  def precision(self):
3      all, hit = 0, 0 # 初始化计数器，all记录推荐物品总数，hit记录命中物品数量
4
5      # 遍历测试集中每个用户
6      for user in self.test:
7          test_items = set(self.test[user]) # 获取该用户在测试集中的真实物品集合
8          rank = self.recs[user] # 获取推荐给该用户的推荐列表
9
10         # 遍历推荐列表中的每个物品
11         for item, score in rank:
12             if item in test_items: # 如果推荐物品在测试集真实物品中
13                 hit += 1 # 命中计数加1
14
15         all += len(rank) # 累加推荐物品的总数
16
17     # 计算精确率，hit/all，并保留两位小数
18     return round(hit / all * 100, 2)
19
```

```
1  // 定义精确率计算函数
2  double precision(const unordered_map<string, vector<int>>& test,
3                  const unordered_map<string, vector<pair<int, double>>&
4                  recs) {
5      int all = 0; // 推荐物品总数
6      int hit = 0; // 命中物品数量
7
8      // 遍历测试集中每个用户
9      for (const auto& [user, test_items_vec] : test) {
10         unordered_set<int> test_items(test_items_vec.begin(),
11         test_items_vec.end()); // 转为集合以加速查找
12         const auto& rank = recs.at(user); // 获取推荐给用户的推荐列表
13
14         // 遍历推荐列表中的每个物品
15         for (const auto& [item, score] : rank) {
16             if (test_items.find(item) != test_items.end()) { // 如果推荐物品
17                 // 命中计数加1
18                 hit++;
19             }
20         }
21
22         all += rank.size(); // 累加推荐物品总数
23     }
24
25     // 计算精确率，避免除以0的情况
26     return all > 0 ? round((double(hit) / all) * 100 * 100) / 100 : 0.0;
27 }
```

```

1  /**
2   * 计算精确率
3   *
4   * @param test 测试集，格式为 Map<String, List<Integer>>
5   * @param recs 推荐列表，格式为 Map<String, List<Pair<Integer, Double>>>
6   * @return 精确率（保留两位小数）
7   */
8  public static double calculatePrecision(Map<String, List<Integer>>
test,
9                                          Map<String,
List<Map.Entry<Integer, Double>>> recs) {
10      int hit = 0; // 命中物品数量
11      int all = 0; // 推荐物品总数
12
13      // 遍历测试集中每个用户
14      for (String user : test.keySet()) {
15          Set<Integer> testItems = new HashSet<>(test.get(user)); // 转换
为集合以便快速查找
16          List<Map.Entry<Integer, Double>> rank = recs.get(user); // 推荐
列表
17
18          // 遍历推荐列表中的每个物品
19          if (rank != null) {
20              for (Map.Entry<Integer, Double> entry : rank) {
21                  if (testItems.contains(entry.getKey())) {
22                      hit++; // 如果命中，计数加1
23                  }
24              }
25              all += rank.size(); // 累加推荐物品总数
26          }
27      }
28
29      // 计算精确率，防止除以0
30      return all > 0 ? Math.round((hit * 100.0 / all) * 100.0) / 100.0 :
0.0;
31  }
32

```

2. 召回率 (Recall)

- **定义：**
测试集中的物品中，实际被推荐系统成功推荐的物品比例。
反映推荐结果的覆盖情况。
- **公式：**

$$\text{Recall} = \frac{\text{命中数}}{\text{测试集总数}}$$

3. 覆盖率 (Coverage)

- **定义：**
推荐系统能为用户推荐的物品种类占总物品种类的比例。
衡量推荐结果的多样性。
- **公式：**

$$\text{Coverage} = \frac{\text{推荐物品种类数}}{\text{总物品种类数}}$$

4. 新颖度 (Popularity)

- 定义:

衡量推荐列表中物品的流行程度，越多长尾物品则新颖度越高。
通过流行度（如被用户评分的次数）计算推荐物品的平均流行度。

- 公式:

$$\text{Popularity} = \frac{\sum \log(1 + \text{物品流行度})}{\text{推荐物品总数}}$$

(取对数是为缓解长尾分布对结果的影响)

3. 实现余弦相似度

4. 测试基于余弦相似度的推荐

示例代码

```
1 class Experiment():
2
3     def __init__(self, M, K, N, fp='../dataset/ml-1m/ratings.dat',
4         rt='UserCF'):
5         '''
6         :params: M, 进行多少次实验
7         :params: K, TopK相似用户的个数
8         :params: N, TopN推荐物品的个数
9         :params: fp, 数据文件路径
10        :params: rt, 推荐算法类型, 这里只使用 'UserCF' 作为算法
11        '''
12        # 初始化实验所需的参数
13        self.M = M # 实验的次数
14        self.K = K # TopK相似用户数
15        self.N = N # TopN推荐物品数
16        self.fp = fp # 数据集文件路径
17        self.rt = rt # 推荐算法类型, 默认'UserCF'
18        self.alg = {'UserCF': UserCF}
19
20        # 定义单次实验
21        @timmer
22        def worker(self, train, test):
23            '''
24            :params: train, 训练数据集
25            :params: test, 测试数据集
26            :return: 各指标的值
27            '''
28            # 获取推荐算法并执行推荐
29            getRecommendation = self.alg[self.rt](train, self.K, self.N)
30            # 计算指标
31            metric = Metric(train, test, getRecommendation)
32            return metric.eval() # 返回计算结果
33
34        # 多次实验取平均
35        @timmer
36        def run(self):
```

```

37         执行M次实验并计算各项指标的平均值
38         :return: 输出各指标的平均值
39         ...
40         # 初始化存储结果的字典
41         metrics = {'Precision': 0, 'Recall': 0,
42                     'Coverage': 0, 'Popularity': 0}
43         dataset = Dataset(self.fp) # 加载数据集
44         for ii in range(self.M): # 执行M次实验
45             train, test = dataset.splitData(self.M, ii) # 切分数据集为训练集和
测试集
46             print('Experiment {}:'.format(ii))
47             metric = self.worker(train, test) # 执行单次实验并计算指标
48             # 累加各个指标的值
49             metrics = {k: metrics[k] + metric[k] for k in metrics}
50             # 计算每个指标的平均值
51             metrics = {k: metrics[k] / self.M for k in metrics}
52             # 输出平均结果
53             print('Average Result (M={}, K={}, N={}): {}'.format(self.M,
self.K, self.N, metrics))
54         # 3. UserCF实验
55         M, N = 8, 10
56         for K in [5, 10, 20, 40, 80, 160]:
57             cf_exp = Experiment(M, K, N,)
58             cf_exp.run()

```

```

1  import java.util.*;
2
3  public class Experiment {
4
5      private int M, K, N;
6      private String fp, rt;
7      private Map<String, UserCF> alg;
8
9      // 构造函数
10     public Experiment(int M, int K, int N, String fp, String rt) {
11         this.M = M;
12         this.K = K;
13         this.N = N;
14         this.fp = fp;
15         this.rt = rt;
16         this.alg = new HashMap<>();
17         this.alg.put("UserCF", new UserCF()); // 只保留UserCF算法
18     }
19
20     // 执行单次实验
21     public Map<String, Double> worker(Map<Integer, Map<Integer, Double>>
train,
22                                     Map<Integer, Map<Integer, Double>>
test) {
23         // 获取推荐算法并执行推荐
24         UserCF getRecommendation = alg.get(rt);
25         Metric metric = new Metric(train, test, getRecommendation);
26         return metric.eval(); // 返回计算结果
27     }
28
29     // 多次实验取平均
30     public void run() {

```

```

31     Map<String, Double> metrics = new HashMap<>();
32     metrics.put("Precision", 0.0);
33     metrics.put("Recall", 0.0);
34     metrics.put("Coverage", 0.0);
35     metrics.put("Popularity", 0.0);
36
37     Dataset dataset = new Dataset(fp); // 加载数据集
38     for (int ii = 0; ii < M; ii++) { // 执行M次实验
39         Map<Integer, Map<Integer, Double>> train, test;
40         train, test = dataset.splitData(M, ii); // 切分数据集为训练集和测
    试集
41
42         System.out.println("Experiment " + ii + ":");
43         Map<String, Double> metric = worker(train, test); // 执行单次实
    验并计算指标
44
45         // 累加各个指标的值
46         for (String key : metrics.keySet()) {
47             metrics.put(key, metrics.get(key) + metric.get(key));
48         }
49
50         // 计算每个指标的平均值
51         for (String key : metrics.keySet()) {
52             metrics.put(key, metrics.get(key) / M);
53         }
54
55         // 输出平均结果
56         System.out.println("Average Result (M=" + M + ", K=" + K + ", N=" +
    N + "): " + metrics);
57     }
58
59     public static void main(String[] args) {
60         int M = 8;
61         int N = 10;
62         for (int K : new int[]{5, 10, 20, 40, 80, 160}) {
63             Experiment cf_exp = new Experiment(M, K, N, "../dataset/ml-
    1m/ratings.dat", "UserCF");
64             cf_exp.run();
65         }
66     }
67 }

```

```

1  #include <iostream>
2  #include <vector>
3  #include <unordered_map>
4  #include <functional>
5  #include <string>
6
7  using namespace std;
8
9  class Experiment {
10 public:
11     // 构造函数
12     Experiment(int M, int K, int N, string fp = "../dataset/ml-
    1m/ratings.dat", string rt = "UserCF") {
13         this->M = M;
14         this->K = K;

```

```

15     this->N = N;
16     this->fp = fp;
17     this->rt = rt;
18     this->alg["UserCF"] = &UserCF; // 只保留UserCF算法
19 }
20
21 // 执行单次实验
22 unordered_map<string, double> worker(unordered_map<int,
unordered_map<int, double>> &train,
23                                     unordered_map<int,
unordered_map<int, double>> &test) {
24     // 获取推荐算法并执行推荐
25     auto getRecommendation = alg[rt];
26     Metric metric(train, test, getRecommendation);
27     return metric.eval(); // 返回计算结果
28 }
29
30 // 多次实验取平均
31 void run() {
32     unordered_map<string, double> metrics = {"Precision", 0},
{"Recall", 0},
33                                           {"Coverage", 0},
{"Popularity", 0}};
34     Dataset dataset(fp); // 加载数据集
35     for (int ii = 0; ii < M; ii++) { // 执行M次实验
36         auto [train, test] = dataset.splitData(M, ii); // 切分数据集为训
练集和测试集
37         cout << "Experiment " << ii << ":" << endl;
38         auto metric = worker(train, test); // 执行单次实验并计算指标
39         // 累加各个指标的值
40         for (auto &pair : metrics) {
41             pair.second += metric[pair.first];
42         }
43     }
44     // 计算每个指标的平均值
45     for (auto &pair : metrics) {
46         pair.second /= M;
47     }
48     // 输出平均结果
49     cout << "Average Result (M=" << M << ", K=" << K << ", N=" << N <<
"): ";
50     for (const auto &pair : metrics) {
51         cout << pair.first << ": " << pair.second << " ";
52     }
53     cout << endl;
54 }
55
56 private:
57     int M, K, N; // 实验的次数、相似用户数和推荐物品数
58     string fp, rt; // 数据集路径和推荐算法类型
59     unordered_map<string, function<UserCF>> alg; // 存储推荐算法
60 };
61
62 int main() {
63     int M = 8;
64     int N = 10;
65     for (int K : {5, 10, 20, 40, 80, 160}) {
66         Experiment cf_exp(M, K, N);

```

```
67         cf_exp.run();
68     }
69     return 0;
70 }
71
```

结束

展示最终结果，并介绍你对基于用户的协同过滤算法的理解。